

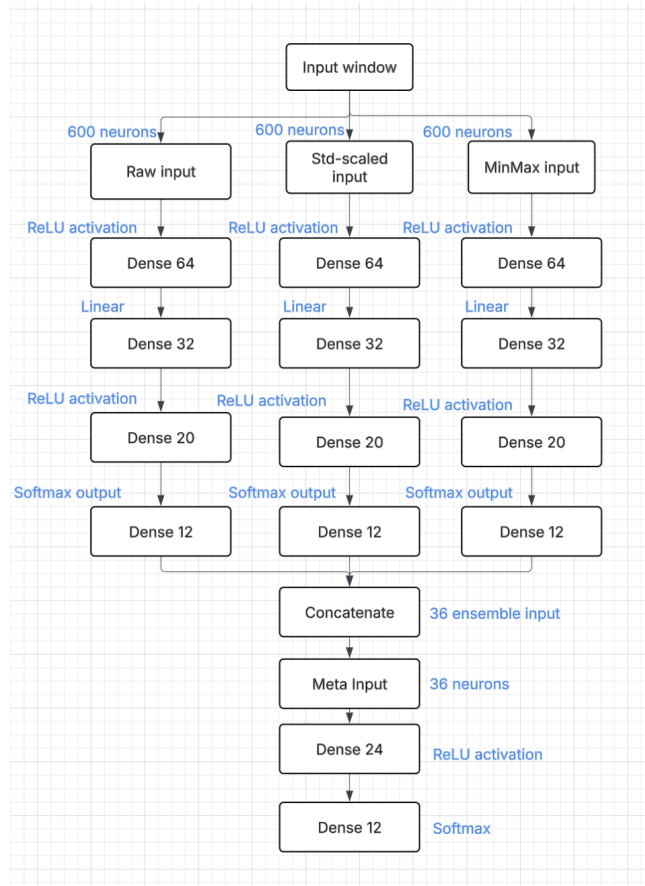
EE446 Lab8
2268225
Momoka Sakamoto

Question 1: Model architecture and ensemble flow

Draw a complete diagram of the full ensemble pipeline. Your diagram should include enough detail for someone to reconstruct the model architecture from your drawing.

Include the following details:

- The input window size and number of input neurons. In this notebook, each input window has 100 time steps and 6 IMU features, so the flattened input dimension is 600.
- The three input branches: raw input, standard-scaled input, and min-max-scaled input.
- The encoder architecture in each branch, including the number of hidden layers, number of neurons in each layer, and activation function used in each layer.
- The classifier architecture in each branch, including the number of input neurons, hidden-layer neurons, output neurons, and activation functions.
- The number of models used in the ensemble and how they are connected.
- The size of each branch output and how the three outputs are concatenated before being passed into the stacked meta-classifier.
- The stacked meta-classifier architecture, including input size, hidden layer size, output size, and activation functions.



Question 2: Pruning and QAT methodology

Explain how this notebook compresses the models for TinyML deployment.

In your answer, describe:

- How pruning is applied and what sparsity schedule is used.
- Why the pruning wrappers are stripped before QAT.
- How QAT is applied after pruning.
- Why the notebook uses an additional mask-enforcement step during and after QAT instead of using only a simple pruning-then-QAT pipeline.
- How the final model is converted to an int8 TFLite model and why representative data is needed.
- How pruning and int8 quantization help when deploying to a resource-constrained device such as the Arduino Nano 33 BLE Sense.

Pruning: Each model is wrapped with `prune_low_magnitude` using PolynomialDecay schedule that ramps sparsity from 10% to 80% over 500 steps across 5 training epochs. The

UpdatePruningStep callback zeros out the lowest-magnitude weights gradually so the model has time to recover accuracy as sparsity increases.

Why wrappers are stripped before QAT: The pruning wrappers are incompatible with `quantize_model`, which expects plain Keras layer types. `strip_pruning` removes the wrapper scaffolding and leaves a standard model where pruned weights are simply zero.

QAT after pruning: `quantize_model` wraps each layer in a `QuantizeWrapper` that inserts fake-quantization nodes simulating 8-bit arithmetic during the forward pass. Fine-tuning for 5 epochs lets the model learn to tolerate quantization noise before real integer conversion.

Why mask enforcement is needed: QAT fine-tuning reintroduces gradients through all parameters, including the zeroed-out weights, which would undo the pruning.

int8 TFLite conversion and representative data: The converter is set to full-integer mode. Weight ranges can be computed statically, but activation ranges require real data to measure. The `representative_dataset` generator passes 200 training samples through the model so the converter can determine accurate per-tensor int8 scale factors for every activation.

Deployment benefit on the Arduino Nano 33 BLE Sense: The board has only 256 KB SRAM and 1 MB flash. Int8 quantization shrinks each weight from 4 bytes to 1, reducing model size. Pruning at 80% sparsity removes additional dead weight, and sparse CMSIS-NN kernels can skip zero-weight operations entirely. Together, they make running a multi-branch ensemble feasible on hardware where float32 models would never fit.

Question 3: Arduino deployment and real-world behavior

Deploy the quantized model using the provided Arduino sketch for this lab. Place the Arduino Nano 33 BLE Sense flat on a stable surface, open the serial monitor, and observe the predicted activity labels.

In your answer, include:

- A screenshot of the serial monitor output.
- A short explanation of whether the output is acceptable for the resting board condition.
- Any issues you observe, such as unstable predictions, unexpected labels, sensor noise, scaling mismatch, or mismatch between the training data and the real board orientation.
- One or two practical changes that could improve deployment reliability.

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 159 ms, inference 550 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.980469
  lift: 0.019531
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 159 ms, inference 549 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.066406
  lift: 0.933594
Starting inferencing in 2 seconds...
Sampling...
```

The output is acceptable. I tried to detect idle for the first one, and lift in the next one. Both were detected well in this deployment.

Question 4: Deployment Behavior on the Arduino

After deploying the ensemble model on the Arduino Nano 33 BLE Sense / Sense Rev2, did you observe any unexpected prediction behavior? For example, did the model repeatedly predict the same class even when the board was moved differently?

Lift prediction was precise, but idle prediction sometimes didn't work and was sometimes detected as lift.

Briefly explain why this may happen. In your answer, discuss possible causes such as differences between the original training dataset and the live Arduino sensor data, sensor placement, axis orientation, unit conversion, feature scaling, or mismatch between the motion patterns used during training and testing.

I think this happens because when I collected data for the lift class, I didn't tilt the board, which could be similar to the idle data, which made idle detection hard.